INSTRUCTIONS

Please read the instructions carefully before starting the exam:

- You may use the materials on your computer, notebooks, and textbooks.
- You are **NOT allowed** to use electronic devices or any other methods to share data with others.
- Do not arbitrarily disconnect from the **FPTU_EXAMONLINE** network while taking the test. If any network problems occur, immediately notify the exam supervisor for assistance.

In addition to the above conditions, students must also fulfill the following requirements:

1. Use **Visual Studio Code** (VS Code), **MongoDB Compass**, **Postman** or **Thunder Client** (a VSCode extension) tools (offline) to complete the exam
2. Using **NodeJS platform version 22.11.0**
3. Use the browser software available on your computer

This part is very important, please read it carefully and follow the instructions

- You are provided with the data files in the **given** directory.
- You must use the solution sent in advance via email and install the additional library included with the accompanying request.
- You are **not allowed** to download any additional libraries during the test
- Violating any of the above rules will make your exam considered invalid

Instructions for submission

- Before submitting the solution, delete the [**node_modules**] folders in your test to reduce the space, so as to meet the allowed capacity of the **PEA_Client** software
- Use the **PEA_Client** software to browse the **resource** directory and complete the submission process.

System Requirements

Using **MongoDB Compass**, create a database named "**Spr26_B5**". Create **collections** corresponding to the data in the **given** directory. Then, import the data into the collections you created in the database.



*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

[Hình ảnh: MongoDB Compass sidebar hiển thị database "Spr26_B5" expand, chứa các collection (folder icons): departments, employees, locations, projects, worksons.]

Using the given data, build a **RESTFul API** application that fulfills the requirements of the problem:

Note: A score of 0 will be given to the work that

- Not using connection string to MongoDB in the **.env** file
- Not configuring the root path of RESTFul API Web App at: **http://localhost:9999**

All endpoints must:

- Validate input data
- Apply business rules
- Return appropriate HTTP status codes
- Return properly structured JSON responses

Context

This system is a Project Management System managing the following collections: **departments, employees, projects, locations**, and **worksons**.

Students must build the API endpoints using **Express JS, MongoDB,** and **Mongoose**. Each task specifies an API operation to be implemented, covering input validation, business rules, database processing, and proper JSON response formatting.

Students must use **Postman** (offline) or **Thunder Client** to test their APIs.

Question 1 (1.5 points): Retrieve Projects by Department

Objective: Retrieve all projects belonging to a specific department, along with the department manager's information.

Endpoint:

```
GET /api/projects/department/:depId
```

Business Rules:

- Validate depId as ObjectId
- Return 400 if invalid
- Return 404 if department not found
- Populate department manager (managerId → empName)
- Retrieve all projects by depId
- Return:
  - department name
  - manager name
  - list of projects

Expected Response:

- **400 (Bad Request)**

```
{
    "error": "Invalid ObjectId: 65a1000000000000000000X"
}
```

- **404 (Not Found)**

```
{
    "error": "Department with _id='65a100000000000000000004' not found"
}
```

- **200 (OK)**

```
{
    "depName": "Information Technology",
    "manager": "Nguyen Van An",
    "projects": [
        {
            "proId": "65a300000000000000000001",
            "proName": "Smart City System"
        }
    ]
}
```

Question 2 (2.5 points): Retrieve Employees by Project

Objective:

Retrieve all employees participating in a specific project together with their work hours.

Endpoint:

```
GET /api/projects/:proId/employees
```

Business Rules:

- Validate proId (khuyến khích, không bắt buộc theo code hiện tại)
- Return 404 if project not found
- Retrieve data from worksOns
- Populate employee info (empId → empName, empSalary)
- Return:
  - project name
  - employee list with work hours

Expected Response:

- **404 (Not Found):**

```
{
    "error": "Project with _id='65a30000000000000000004' not found"
}
```

- **200 (OK)**: Test with proId="65a30000000000000000001"

```
{
    "project": "Smart City System",
    "employees": [
        {
            "empName": "Tran Thi Binh",
            "empSalary": 1800,
            "workHours": 20
        },
        {
            "empName": "Le Van Cuong",
            "empSalary": 1700,
            "workHours": 25
        }
    ]
}
```

---

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

Question 3 (2.5 points): Project Work Hours Report

Objective: Produce a report of the total working hours and the number of employees for each project.

Endpoint:

```
GET /api/reports/projects/work-hours
```

Business Rules:

- Use Aggregation Pipeline
- Group by proId
- Calculate:
  - total work hours
  - total employees
- Join with:
  - projects collection
  - departments collection
- Return project name and department name

Expected Response:

- **200 (OK):**

```
[
    {
        "totalWorkHours": 48,
        "totalEmployees": 2,
        "projectName": "E-Commerce Platform",
        "departmentName": "Sales and Marketing"
    },
    {
        "totalWorkHours": 45,
        "totalEmployees": 2,
        "projectName": "Smart City System",
```

```
        "departmentName": "Information Technology"
    },
    {
        "totalWorkHours": 10,
        "totalEmployees": 1,
        "projectName": "AI Traffic Control",
        "departmentName": "Research and Development"
    }
]
```

Question 4 (3.5 points): Create Project with Employees

Objective: Create a new project and assign employees to it together with their working hours.

Endpoint:

```
POST /api/projects
```

Request Body:

```
{
    "proName": "Digital Government",
    "depId": "65a1000000000000000000001",
    "location": [
        {"locId": "65a0000000000000000000001"},
        {"locId": "65a0000000000000000000003"}
    ],
    "employees": [
        {
            "empId": "65a2000000000000000000002",
            "workHours": 20
        },
        {
            "empId": "65a2000000000000000000003",
            "workHours": 25
        }
    ]
}
```

Business Rules:

- Validate department exists
- Return 404 if not found
- If employees provided:
  - Validate all empId exist
  - Return 400 if any invalid
- Create project
- Insert records into worksOns
- Return created project info

Expected Response:

- **404 (Not Found):**

```
{
    "error": "Department with _id='65a1000000000000000000005' not found"
}
```

- **201 (Created)**: employees empty, insert into projects only

```
{
    "message": "Project created successfully",
    "newProject": {
        "projectId": "69ba0deab7f537a8b8b5d2f1",
        "projectName": "Digital Government"
    }
}
```

- **400 (Bad Request)**: employees not exist

```
{
    "error": "Some employees do not exist",
    "invalidEmployees": [
        "65a2000000000000000000008",
        "65a2000000000000000000009"
```

```
    ]
}
```

• **201 (Created)**: insert into projects and worksOns

```
{
    "message": "Project created successfully",
    "newProject": {
        "projectId": "69ba0ebeb7f537a8b8b5d2fa",
        "projectName": "Digital Government",
        "worksOns": [
            {
                "_id": "69ba0ebeb7f537a8b8b5d2fd",
                "proId": "69ba0ebeb7f537a8b8b5d2fa",
                "empId": "65a20000000000000000002",
                "workHours": 20
            },
            {
                "_id": "69ba0ebeb7f537a8b8b5d2fe",
                "proId": "69ba0ebeb7f537a8b8b5d2fa",
                "empId": "65a20000000000000000003",
                "workHours": 25
            }
        ]
    }
}
```

---- END ---

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*